

maxentcpp: A C++ reimplementation of Maximum Entropy species distribution modeling for R

Ángel Luis Robles Fernández*

Department of Ecology and Evolutionary Biology,
University of Kansas, Lawrence, KS, United States

May 2026

1 Summary

`maxentcpp` is an R package that provides a high-performance C++17 reimplementation of the Maximum Entropy (Maxent) algorithm for species distribution modeling (SDM). Maxent estimates the geographic distribution of a species from presence-only occurrence records and environmental covariates by finding the probability distribution of maximum entropy subject to constraints derived from the training data [Phillips et al., 2006, 2004]. Since its introduction, Maxent has become one of the most widely cited methods in ecology and conservation biology, with over 13 000 citations [Elith et al., 2011].

The fitted model is a *Gibbs distribution* over geographic space:

$$P(\mathbf{x}) = \frac{1}{Z} \exp\left(\sum_{j=1}^J \lambda_j f_j(\mathbf{x})\right), \quad (1)$$

where f_j are feature transformations of the environmental variables, λ_j are learned weights, and $Z = \sum_{\mathbf{x}} \exp(\sum_j \lambda_j f_j(\mathbf{x}))$ is the normalizing constant (partition function). The optimizer finds the λ_j that maximize the regularized log-likelihood via sequential coordinate ascent with ℓ_1 (lasso) penalties [Phillips et al., 2006, Elith et al., 2011].

The original Maxent software is a Java desktop application [Phillips et al., 2017]. `maxentcpp` translates the complete Maxent 3.4.4 algorithm into C++17 with R bindings via Rcpp [Eddelbuettel and François, 2011] and RcppEigen [Bates and Eddelbuettel, 2013], eliminating the Java Runtime Environment dependency entirely. The package supports all five feature types (linear, quadratic, product, hinge, and threshold), output transformations in raw, logistic, and complementary log-log (cloglog) scales, and a streaming raster evaluation engine that processes environmental layers block-by-block through the `terra` package [Hijmans, 2024], enabling continental-scale projections on commodity hardware without loading entire raster stacks into memory. Model diagnostics include AUC evaluation, permutation importance, response curves, and Multivariate Environmental Similarity Surfaces (MESS) for novelty detection.

2 Statement of need

Maxent is the *de facto* standard for presence-only species distribution modeling [Merow et al., 2013]. However, the original Java implementation presents several practical barriers for modern ecological research workflows:

*Corresponding author. ORCID: 0000-0002-4741-8012. E-mail: a.l.robles.fernandez@gmail.com

1. **Java dependency.** Users must install and configure a compatible JDK. Version conflicts between Java 8, 11, 17, and 21 produce silent failures, and the `rJava` bridge package is one of the most common sources of installation errors in the R ecosystem, particularly on macOS and HPC clusters.
2. **File-based I/O.** Data must be serialized to disk as SWD or ASCII grid files before the Java process can read them, and results must be parsed back from output files. There is no in-memory bridge between R data structures and the Java optimizer.
3. **Scalability.** The Java implementation is single-threaded and GUI-centric. Large-scale studies involving thousands of species, multiple climate scenarios, or ensemble pipelines are bottlenecked by file I/O overhead and the inability to integrate Maxent as a library call within a larger R pipeline.
4. **Maintenance.** The Java Maxent codebase has remained essentially unchanged since version 3.4.4 (released November 2020), with only two active contributors [Phillips et al., 2017]. `maxentcpp` addresses these barriers by providing a native C++/R implementation that installs with a single `install.packages("maxentcpp")` call, operates entirely in memory through R objects, and integrates seamlessly with the `terra` spatial data ecosystem. The target audience is ecologists, conservation biologists, and biogeographers who use Maxent in reproducible, script-based workflows—from individual species analyses to large-scale biodiversity assessments.

3 State of the field

Several R packages provide access to Maxent-family models. `dismo` [Hijmans et al., 2023] is the most widely used R interface to Java Maxent; it wraps `maxent.jar` via `rJava`, inheriting the JDK dependency and file I/O overhead, and is currently in maintenance mode following the transition from `raster` to `terra`. `maxnet` [Phillips, 2024] is a pure-R implementation that uses `glmnet` for elastic-net regularization; while it avoids the Java dependency, it employs a different optimization algorithm (elastic net via coordinate descent on the full feature matrix) rather than the sequential coordinate-ascent optimizer of the original Maxent, which can produce numerically different results, particularly for threshold and hinge features. `ENMeval` [Kass et al., 2021] is a model tuning and evaluation framework that wraps either `dismo::maxent()` or `maxnet::maxnet()`; it does not implement the Maxent algorithm itself. `kuenm` [Cobos et al., 2019] is a calibration and evaluation toolkit that relies on `dismo` or direct Java Maxent calls.

`maxentcpp` was built as a new package rather than contributing to existing projects for three reasons. First, a faithful port of the original Java optimizer (sequential coordinate ascent with ℓ_1 -penalized Newton steps) requires a C++ engine that cannot be expressed through `glmnet`'s API; contributing this to `maxnet` would change its core algorithm. Second, `dismo`'s architecture is tightly coupled to `rJava` and `raster`; the native C++ approach eliminates this dependency chain entirely. Third, the header-based C++ library design of `maxentcpp` enables reuse in other packages or standalone applications beyond R—something not achievable through modifications to existing R-only packages.

4 Software design

4.1 Architecture

`maxentcpp` is organized in three layers (Table 1): a pure C++ core that implements the Maxent algorithm independently of R, an Rcpp bridge that exposes C++ objects to R as external pointers [Eddelbuettel, 2013], and an R interface that handles input validation, spatial data conversion via `terra`, and output formatting.

The high-level `maxent_run()` function provides a one-call workflow mirroring the Java GUI experience, while lower-level functions (`maxent_generate_features()`, `maxent_featured_space()`,

Table 1: Three-layer architecture of `maxentcpp`.

Layer	Key components
C++ core	<code>FeaturedSpace</code> , <code>Sequential</code> , five feature types, <code>BackgroundProvider</code> ; uses Eigen [Guennebaud, 2017]
Rcpp bridge	<code>rcpp_*.cpp</code> external-pointer bindings
R interface	<code>maxent_run()</code> , feature generators, projection, evaluation, diagnostics

`maxent_fit()`, `maxent_project_cloglog()` offer full control over each modeling step.

4.2 Key design decisions

Streaming raster evaluation. Rather than materializing the full background matrix in memory, `maxentcpp` reads `terra::SpatRaster` objects block-by-block through a `BackgroundProvider` abstraction. Each tile is scored by the C++ engine and discarded before the next tile is loaded, allowing projection onto raster stacks larger than available RAM.

Numerical fidelity over algorithmic novelty. Every C++ class is a direct port of the corresponding Java class, preserving the same control flow, regularization logic, and numerical constants (Table 2). This design choice prioritizes backward compatibility: users migrating from Java Maxent obtain identical results. The fidelity contract is enforced by a dedicated companion package, `maxentcppCompTest` [Fernández, 2025], which runs the C++ optimizer and the original Java `density.Sequential` on shared test fixtures and compares per-iteration trajectories of loss, entropy, and λ vectors. On standardized fixtures, agreement reaches $< 10^{-14}$ relative error.

Table 2: Class-level porting map from Java Maxent to `maxentcpp`.

<code>maxentcpp</code> (C++)	Java Maxent original
<code>LinearFeature</code>	<code>density.LinearFeature</code>
<code>QuadraticFeature</code>	<code>density.SquareFeature</code>
<code>ProductFeature</code>	<code>density.ProductFeature</code>
<code>HingeFeature</code>	<code>density.HingeFeature</code>
<code>ThresholdFeature</code>	<code>density.ThresholdFeature</code>
<code>FeaturedSpace</code>	<code>density.FeaturedSpace</code>
<code>Sequential::train()</code>	<code>density.Sequential.run()</code>

Lambda file compatibility. Trained models are serialized in the same `.lambdas` text format used by Java Maxent 3.4.4. Lambda files produced by either implementation can be loaded by the other, ensuring interoperability with existing workflows and archived models.

4.3 Development provenance

The translation followed a phased approach across four publicly accessible repositories:

1. `alrobes/Maxent`—a fork of the original `mrmaxent/Maxent` [Phillips et al., 2017] Java source code, where the architecture was analyzed and each Java class was mapped to a C++ target (193 commits).
2. `alrobes/maxentcpp-devel`—the development repository where the C++ port, Rcpp bindings, R interface, tests, and documentation were iteratively built and refined (34 commits).
3. `alrobes/maxentcppCompTest`—the cross-language fidelity test suite containing Java oracle runners, golden trajectory files, and automated comparison scripts (40 commits).

4. `alrobes/maxentcpp`—the release repository with the clean, CRAN-ready package (47 commits).

5 Research impact statement

`maxentcpp` is designed to serve as a drop-in replacement for Java Maxent in R-based ecological workflows. The package eliminates the most common installation barrier (JRE + `rJava` configuration), enables integration of Maxent into fully scripted and reproducible pipelines, and supports raster-scale projections that are infeasible with file-based I/O.

The package is distributed under the MIT license, with full source code, a pkgdown documentation website (<https://alrobes.github.io/maxentcpp/>), and three vignettes covering a getting-started walkthrough, the mathematical foundations of Maxent features, and a comparative motivation document. Continuous integration via GitHub Actions runs `R CMD check` on Ubuntu, macOS, and Windows across R release and R-devel. The test suite comprises over 20 test files (~4800 lines) covering feature generation, optimization convergence, spatial projection, Java numerical equivalency, model diagnostics, and end-to-end workflows.

By providing a dependency-free, memory-efficient, and numerically faithful Maxent implementation, `maxentcpp` lowers the barrier for large-scale biodiversity assessments, climate-change projections, and conservation prioritization studies that rely on presence-only species distribution models.

6 AI usage disclosure

Generative AI tools were used during the development of `maxentcpp` and the preparation of this manuscript, as detailed below.

Tools used.

- **GitHub Copilot** (GPT-4-based, 2025 version): code scaffolding and boilerplate generation during the initial porting phases in `alrobes/Maxent` and `alrobes/maxentcpp-devel`.
- **Devin** (Cognition AI, 2025–2026): incremental feature implementation, test writing, documentation generation, CRAN compliance fixes, CI configuration, and manuscript drafting.
- **Claude** (Anthropic, 2025–2026): complex refactoring, cross-cutting documentation, and code review.

Nature and scope of assistance. AI tools contributed to: C++ code generation and refactoring from Java source, Rcpp binding scaffolding, R wrapper functions, `testthat` test scaffolding and expansion, `roxygen2` documentation, vignette drafting, pkgdown site configuration, CI workflow setup, CRAN compliance review, and manuscript drafting.

Confirmation of review. All AI-generated code, tests, documentation, and manuscript text were reviewed, edited, and validated by the human author (Á. L. Robles Fernández), who made all core design decisions including the algorithm translation strategy, API design, numerical fidelity requirements, and the phased development architecture. The full development history is publicly available in the four repositories listed above.

Acknowledgements

The author thanks Steven J. Phillips, Robert P. Anderson, and Robert E. Schapire for creating the original Maxent software and making the source code publicly available under an MIT license.

The `Rcpp`, `RcppEigen`, and `terra` package maintainers are gratefully acknowledged for providing the infrastructure that makes `maxentcpp` possible.

References

- Douglas Bates and Dirk Eddelbuettel. Fast and elegant numerical linear algebra using the RcppEigen package. *Journal of Statistical Software*, 52(5):1–24, 2013. doi: 10.18637/jss.v052.i05.
- Marlon E. Cobos, A. Townsend Peterson, Narayani Barve, and Luis Osorio-Olvera. kuenm: an R package for detailed development of ecological niche models using Maxent. *PeerJ*, 7:e6281, 2019. doi: 10.7717/peerj.6281.
- Dirk Eddelbuettel. *Seamless R and C++ Integration with Rcpp*. Springer, New York, 2013. doi: 10.1007/978-1-4614-6868-4.
- Dirk Eddelbuettel and Romain François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011. doi: 10.18637/jss.v040.i08.
- Jane Elith, Steven J. Phillips, Trevor Hastie, Miroslav Dudík, Yung En Chee, and Colin J. Yates. A statistical explanation of MaxEnt for ecologists. *Diversity and Distributions*, 17(1):43–57, 2011. doi: 10.1111/j.1472-4642.2010.00725.x.
- Ángel Luis Robles Fernández. maxentcppCompTest: Cross-language fidelity tests for maxentcpp, 2025. URL <https://github.com/alroble/maxentcppCompTest>.
- Gaël Guennebaud, Benoît Jacob, et al. Eigen v3, 2010. URL <https://eigen.tuxfamily.org>.
- Robert J. Hijmans. *terra: Spatial Data Analysis*, 2024. URL <https://CRAN.R-project.org/package=terra>. R package version 1.7-78.
- Robert J. Hijmans, Steven Phillips, John Leathwick, and Jane Elith. *dismo: Species Distribution Modeling*, 2023. URL <https://CRAN.R-project.org/package=dismo>. R package version 1.3-14.
- Jamie M. Kass, Robert Muscarella, Peter J. Galante, Corentin L. Bohl, Gonzalo E. Pinilla-Buitrago, Robert A. Boria, Mariano Soley-Guardia, and Robert P. Anderson. ENMeval 2.0: Redesigned for customizable and reproducible modeling of species’ niches and distributions. *Methods in Ecology and Evolution*, 12(9):1602–1608, 2021. doi: 10.1111/2041-210X.13628.
- Cory Merow, Matthew J. Smith, and John A. Silander, Jr. A practical guide to MaxEnt for modeling species’ distributions: what it does, and why inputs and settings matter. *Ecography*, 36(10):1058–1069, 2013. doi: 10.1111/j.1600-0587.2013.07872.x.
- Steven J. Phillips. *maxnet: Fitting ‘Maxent’ Species Distribution Models with ‘glmnet’*, 2024. URL <https://CRAN.R-project.org/package=maxnet>. R package version 0.1.4.
- Steven J. Phillips, Miroslav Dudík, and Robert E. Schapire. A maximum entropy approach to species distribution modeling. In *Proceedings of the Twenty-First International Conference on Machine Learning*, ICML ’04, pages 655–662, New York, NY, USA, 2004. ACM. doi: 10.1145/1015330.1015412.
- Steven J. Phillips, Robert P. Anderson, and Robert E. Schapire. Maximum entropy modeling of species geographic distributions. *Ecological Modelling*, 190(3–4):231–259, 2006. doi: 10.1016/j.ecolmodel.2005.03.026.

Steven J. Phillips, Robert P. Anderson, Miroslav Dudík, Robert E. Schapire, and Mary E. Blair.
Opening the black box: an open-source release of Maxent. *Ecography*, 40(7):887–893, 2017.
doi: 10.1111/ecog.03049.